深度学习
第 4 章　反向传播算法

——神经网络是怎么”知道”自己哪里错了，又是怎么改正的?

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

反向传播是深度学习的心脏——没有它，网络就无法从错误中学习。本章 6 道题从计算图的”画法”一路打通到完整代码实现，把这个被误解最多的算法彻底讲清楚：

- **Q4-01（计算图阅读）**：前向传播的计算图构建——给定一个数学表达式，画出对应的计算图，标注中间变量，为反向传播打基础。
- **Q4-02（链式法则推导）**：链式法则手动推导——从复合函数的角度，一步步写出梯度沿计算图反向流动的公式，把链式法则从”记公式”变成”会计算”。
- **Q4-03（节点反向传播）**：加法节点与乘法节点的反向传播——两类最基本的节点，梯度各自怎么传？填表 + 数值验证。
- **Q4-04（激活层推导）**：ReLU 和 Sigmoid 的反向传播——推导两种激活函数的梯度公式，说明为什么 ReLU 的梯度”更干净”。
- **Q4-05（Affine 层推导）**：矩阵形式的反向传播——全连接层（Affine 层）在批量输入下，权重矩阵 W、偏置 b、输入 X 的梯度各是什么形状？如何推导?
- **Q4-06（综合代码实现）**：完整反向传播代码追踪——阅读一个含 Affine + ReLU + Softmax-CrossEntropy 的两层网络实现，追踪梯度在各层之间的流动。

1　第 4 章　反向传播算法

上一章我们理解了神经网络的前向传播——数据从输入层经过若干变换流向输出层，得到预测值，和标签比较，得到一个 Loss。

但”知道 Loss 有多大”只完成了一半。真正的学习发生在之后：

- **Loss 对哪个权重的影响最大？**——这些权重需要调整最多；
- **Loss 对哪个权重的影响接近零？**——这些权重暂时不用动；
- **应该让权重变大还是变小？**——朝着让 Loss 减小的方向走。

回答这三个问题，需要计算 Loss 对每个参数的偏导数（梯度）。对于一个含数百层、数百万参数的网络，手推梯度几乎不可能——**反向传播算法**就是高效自动完成这件事的核心工具。

先建立一个总感觉：反向传播是什么？

把神经网络比作一条装配流水线，Loss 是最终的”质检报告”。

- **前向传播** = 零件从左到右经过每道工序，最终组装出产品；
- **Loss** = 质检员发现产品有缺陷，给出一个”不合格分数”；
- **反向传播** = 质检报告从右向左传递：先告诉最后一道工序”你的错误占了多少比例”，再告诉倒数第二道工序，……一直追溯到源头；
- **梯度下降** = 每道工序根据自己被”归责”的比例，调整操作参数，下次少出错。

反向传播的本质就是：**用链式法则，把”最终错误”按照因果比例，分配给每一个参数。**

Q4-01 计算图的构建与前向传播

题目描述：

文档第 4.1 节介绍了计算图的概念。计算图是反向传播的" 地图"——把数学表达式画成有向图，每个节点表示一次运算，每条边表示数据流动的方向。

考虑以下前向计算过程（一个简化的单样本损失计算）：

$$z = x \cdot w + b, \quad \hat{y} = \sigma(z), \quad L = -\big(y \log \hat{y} + (1-y)\log(1-\hat{y})\big)$$

其中 $x = 2$，$w = 0.5$，$b = 1$，$y = 1$（真实标签），σ 为 Sigmoid 函数。

问题：

(1) 按照前向计算顺序，依次计算以下中间变量的数值（结果保留三位小数）：

a. $z = x \cdot w + b =$ _____

b. $\hat{y} = \sigma(z) = \dfrac{1}{1+e^{-z}} =$ _____

c. $L = -\big(y \log \hat{y} + (1-y)\log(1-\hat{y})\big) =$ _____

(2) 计算图中每个节点只做**一种基本运算**。将上述计算分解，列出从输入 (x, w, b, y) 到 L 所经历的所有基本运算节点，并标注每个节点的输入和输出：

节点编号	运算类型	输入变量	输出变量	输出数值
1	乘法	x，w	$t_1 = x \cdot w$	_____
2	加法	t_1，b	$t_2 = t_1 + b$	_____
3	Sigmoid	t_2	$\hat{y} = \sigma(t_2)$	_____
4	对数	$\hat{y}$	$t_3 = \log \hat{y}$	_____
5	乘法	y，t_3	$t_4 = y \cdot t_3$	_____
6	取负	t_4	$L = -t_4$	_____

（注：此处简化为 $y = 1$ 时的情况，省略了 $(1-y)\log(1-\hat{y})$ 项）

(3) 计算图有两个核心优势：**模块化**和**局部计算**。用一句话解释" 局部计算" 是什么意思，以及它为什么让反向传播变得高效。

来源溯源：[文档第 4 章 4.1 计算图]

小白必读：为什么要把计算”画”成图？

- **计算图的本质**：把一个复杂的数学表达式拆解成一系列”只做一件事”的小节点，每个节点知道自己的输入是什么、输出是什么、以及这个运算的导数是多少。
- **为什么要拆？**拆成图之后，反向传播可以”逐节点”地传递梯度，每个节点只需要知道”上游传来的梯度”和”自己这个运算的局部导数”，乘一下就得到传给下游的梯度——不需要知道整个网络的全局结构。
- **计算图 = 自动微分的蓝图**：PyTorch 的 `autograd` 背后做的事，就是在前向传播时悄悄构建这张图，在 `backward()` 时沿图反向遍历，自动计算每个节点的梯度。

参考答案与详细解析

(1) 中间变量数值计算：

a. $z = x \cdot w + b = 2 \times 0.5 + 1 = 1 + 1 = \boxed{2.000}$

b. $\hat{y} = \sigma(2) = \dfrac{1}{1+e^{-2}} = \dfrac{1}{1+0.135} \approx \boxed{0.880}$

c. $y = 1$ 时，$(1-y)\log(1-\hat{y}) = 0$，故：

$$L = -\log(0.880) \approx -(-0.128) = \boxed{0.128}$$

(2) 计算图节点数值：

节点编号	运算类型	输入变量	输出变量	输出数值
1	乘法	$x = 2,\ w = 0.5$	t_1	1.000
2	加法	$t_1 = 1,\ b = 1$	t_2	2.000
3	Sigmoid	$t_2 = 2$	$\hat{y}$	0.880
4	对数	$\hat{y} = 0.880$	t_3	-0.128
5	乘法	$y = 1,\ t_3 = -0.128$	t_4	-0.128
6	取负	$t_4 = -0.128$	L	0.128

(3) 局部计算的含义：

局部计算是指：计算图中每个节点在前向传播时只需要关注自己的输入，在反向传播时只需要知道上游传来的梯度和自己的局部导数（对自身输入的偏导数）——与网络其他部分的结构完全无关。这使得反向传播可以**分节点独立计算**、**模块化复用**，无论网络有多深多宽，每个节点的计算复杂度都是常数，整体复杂度仅与网络规模线性相关，远比直接对整个网络展开求导高效。

Q4-02 链式法则：梯度沿计算图反向流动

题目描述：

文档第 4.2 节介绍了链式法则（Chain Rule）——反向传播的数学基础。链式法则说的是：对于复合函数 $L = f(g(x))$，有：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial g} \cdot \frac{\partial g}{\partial x}$$

考虑一个三层复合函数（对应一条三节点的计算图路径）：

$$t_1 = 3x^2, \quad t_2 = t_1 + 5, \quad L = 2t_2$$

问题：

(1) 用链式法则，推导 $\frac{\partial L}{\partial x}$。写出完整的推导过程，分三步展开：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial t_2} \cdot \frac{\partial t_2}{\partial t_1} \cdot \frac{\partial t_1}{\partial x}$$

(2) 在计算图中，反向传播从 L 开始，将梯度逐节点向左传递。设”上游梯度”为从右侧传来的梯度值，填写下表（取 $x = 1$ 时的数值）：

节点	该节点的局部导数	上游梯度（从右侧传来）	传出的梯度（上游 × 局部）
$L = 2t_2$	$\frac{\partial L}{\partial t_2} =$ __	1（Loss 对自身的导数）	_____
$t_2 = t_1 + 5$	$\frac{\partial t_2}{\partial t_1} =$ __	_____	_____
$t_1 = 3x^2$	$\frac{\partial t_1}{\partial x} =$ __	_____	_____

(3) 验证：直接计算 $L = 2(3x^2 + 5) = 6x^2 + 10$，对 x 求导，在 $x = 1$ 处的值是多少？与上表最终结果是否一致？

(4) 当计算图出现**分支**（即一个节点的输出同时被多个下游节点使用）时，该节点的梯度应该如何处理？（提示：考虑 $z = x + x^2$ 时，x 出现了两次）

来源溯源：[文档第 4 章 4.2 链式法则]

小白必读：链式法则为什么叫”链式”？

- **链式**的含义：梯度像一条链条上的力，从最终输出端（Loss）出发，一环一环地向后传递，每经过一个节点就乘以该节点的局部导数，直到传递到每一个参数。
- **关键洞察**：每个节点不需要知道”Loss 从哪来”，只需要知道”上游告诉我梯度是多少”，然后用自己的局部导数乘一下，传给下游即可。这就是为什么可以把整个网络拆成独立模块——每个模块只实现自己的 `forward`（计算输出）和 `backward`（乘以局部导数传递梯度）。
- **链式法则的核心公式**：

$$\frac{\partial L}{\partial x} = \underbrace{\frac{\partial L}{\partial y}}_{\text{上游梯度}} \times \underbrace{\frac{\partial y}{\partial x}}_{\text{局部导数}}$$

参考答案与详细解析

(1) 链式法则推导：

$$\frac{\partial L}{\partial t_2} = 2, \quad \frac{\partial t_2}{\partial t_1} = 1, \quad \frac{\partial t_1}{\partial x} = 6x$$

$$\frac{\partial L}{\partial x} = 2 \times 1 \times 6x = 12x$$

(2) 反向传播逐节点填表（$x = 1$）：

节点	局部导数	上游梯度	传出的梯度
$L = 2t_2$	$\partial L/\partial t_2 = 2$	1	$2 \times 1 = 2$
$t_2 = t_1 + 5$	$\partial t_2/\partial t_1 = 1$	2	$1 \times 2 = 2$
$t_1 = 3x^2$	$\partial t_1/\partial x = 6x = 6$	2	$6 \times 2 = \mathbf{12}$

(3) 直接求导验证：
$L = 6x^2 + 10$，$\dfrac{\partial L}{\partial x} = 12x$，在 $x = 1$ 处：$\dfrac{\partial L}{\partial x}\Big|_{x=1} = 12$。
与表中最终结果 **12** 完全一致。✓

(4) 分支节点的梯度处理——梯度累加：
当一个节点的输出同时被多个下游节点使用时，该节点从多个下游接收梯度，应将所有来源的梯度**相加**。
例如 $L = z = x + x^2$，可以理解为 x 同时进入了加法节点（贡献 1）和平方节点（贡献 $2x$），两条路径的梯度相加：

$$\frac{\partial L}{\partial x} = 1 + 2x$$

在计算图中，这对应”两条从下游传来的梯度箭头在 x 节点处汇合，执行求和”。

反向传播两大规则速记

- **串联节点**（单路径）：梯度 = 上游梯度 × 局部导数（链式相乘）
- **分支节点**（多路径汇合）：梯度 = 所有下游传来梯度之**和**（梯度叠加）

Q4-03 加法节点与乘法节点的反向传播

题目描述：

文档第 4.3 节指出，任意复杂的计算图都可以分解为加法节点和乘法节点的组合。掌握这两种最基本节点的反向传播规则，是理解整个反向传播算法的基石。

问题：

(1) **加法节点**：设 $z = x + y$，上游传来的梯度为 $\dfrac{\partial L}{\partial z} = \delta$。
推导 $\dfrac{\partial L}{\partial x}$ 和 $\dfrac{\partial L}{\partial y}$，并用一句话总结加法节点反向传播的规律。

(2) **乘法节点**：设 $z = x \cdot y$，上游传来的梯度为 $\dfrac{\partial L}{\partial z} = \delta$。
推导 $\dfrac{\partial L}{\partial x}$ 和 $\dfrac{\partial L}{\partial y}$，并用一句话总结乘法节点反向传播的规律。

(3) **数值验证**：已知计算图中某乘法节点，前向传播时输入为 $x = 3$，$y = -2$，上游梯度 $\delta = 4$。计算传给 x 和 y 的梯度。

(4) **综合练习**：对于表达式 $L = (x + y) \cdot z$，其中 $x = 1$，$y = 2$，$z = 3$：

a. 写出前向传播的计算过程（设中间变量 $s = x + y$）；

b. 从 $\dfrac{\partial L}{\partial L} = 1$ 出发，逐节点反向传播，求出 $\dfrac{\partial L}{\partial x}$，$\dfrac{\partial L}{\partial y}$，$\dfrac{\partial L}{\partial z}$；

c. 用直接求偏导数的方法验证答案（$L = xz + yz$，分别对 x、y、z 求偏导）。

来源溯源：[文档第 4 章 4.3 反向传播]

生活比喻：加法节点和乘法节点的梯度像什么？

- **加法节点 = 快递分拣中心**：一个包裹到达，原封不动地分发给两个收件人（x 和 y）。不管包裹大小，两边收到的都是同样大小的”副本”。梯度原样广播到所有输入。
- **乘法节点 = 权重放大器**：上游说”整体需要调 δ 倍”，但对 x 的调整量由 y 的大小决定（$\delta \cdot y$），对 y 的调整量由 x 的大小决定（$\delta \cdot x$）——谁对结果贡献越大（值越大），谁对另一方的梯度放大越多。
- **乘法节点的反直觉之处**：梯度中出现了**另一方**的值，这意味着如果某个输入值很大，对应的梯度也会很大——这正是为什么深度网络中权重初始化过大会引发梯度爆炸。

参考答案与详细解析

(1) 加法节点反向传播：

$z = x + y$，局部导数为 $\dfrac{\partial z}{\partial x} = 1$，$\dfrac{\partial z}{\partial y} = 1$。

由链式法则：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x} = \delta \cdot 1 = \delta$$

$$\frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial y} = \delta \cdot 1 = \delta$$

规律：加法节点将上游梯度**原样广播**给所有输入，不做任何放大或缩小。

(2) 乘法节点反向传播：

$z = x \cdot y$，局部导数为 $\dfrac{\partial z}{\partial x} = y$，$\dfrac{\partial z}{\partial y} = x$。

$$\frac{\partial L}{\partial x} = \delta \cdot y, \quad \frac{\partial L}{\partial y} = \delta \cdot x$$

规律：乘法节点将上游梯度乘以**另一方的前向传播值**后传出——x 的梯度含 y，y 的梯度含 x。

(3) 数值验证（$x = 3$，$y = -2$，$\delta = 4$）：

$$\frac{\partial L}{\partial x} = \delta \cdot y = 4 \times (-2) = \boxed{-8}$$
$$\frac{\partial L}{\partial y} = \delta \cdot x = 4 \times 3 = \boxed{12}$$

(4) 综合练习（$L = (x + y) \cdot z$，$x = 1$，$y = 2$，$z = 3$）：

(a) 前向传播：$s = x + y = 3$，$L = s \cdot z = 9$

(b) 反向传播：

从 $L = s \cdot z$ 出发，$\dfrac{\partial L}{\partial L} = 1$：

$$\frac{\partial L}{\partial s} = 1 \cdot z = 3, \quad \frac{\partial L}{\partial z} = 1 \cdot s = 3$$

从 $s = x + y$ 出发，上游梯度为 3：

$$\frac{\partial L}{\partial x} = 3 \cdot 1 = \boxed{3}, \quad \frac{\partial L}{\partial y} = 3 \cdot 1 = \boxed{3}, \quad \frac{\partial L}{\partial z} = \boxed{3}$$

(c) 直接求偏导验证：$L = xz + yz$

$$\frac{\partial L}{\partial x} = z = 3 \checkmark, \quad \frac{\partial L}{\partial y} = z = 3 \checkmark, \quad \frac{\partial L}{\partial z} = x + y = 3 \checkmark$$

三者完全一致。

Q4-04 激活层的反向传播：ReLU 与 Sigmoid

题目描述：

文档第 4.4 节推导了 ReLU 和 Sigmoid 两种激活函数的反向传播公式。激活层是计算图中的”非线性节点”，其梯度的”干净程度”直接影响梯度能否顺利流向底层。

问题：

(1) **ReLU 的反向传播**：ReLU 定义为：

$$y = \mathrm{ReLU}(x) = \max(0, x)$$

推导 $\dfrac{\partial L}{\partial x}$（用上游梯度 $\delta = \dfrac{\partial L}{\partial y}$ 表示），并解释为什么 ReLU 的反向传播可以用一个简单的”开关”来描述。

(2) **Sigmoid 的反向传播**：Sigmoid 定义为 $y = \sigma(x) = \dfrac{1}{1+e^{-x}}$，其导数有一个优美的形式：

$$\frac{d\sigma}{dx} = \sigma(x)\big(1-\sigma(x)\big) = y(1-y)$$

推导 $\dfrac{\partial L}{\partial x}$（用 y 和 δ 表示），并计算：当 $y = 0.880$（即上题 Q4-01 中的 $\hat{y}$），$\delta = 1$ 时，$\dfrac{\partial L}{\partial x}$ 的值（保留三位小数）。

(3) **梯度”干净程度”对比**：分析并填写下表：

激活函数	反向传播公式	梯度特点（从防止梯度消失角度分析）
ReLU	$\dfrac{\partial L}{\partial x} = \begin{cases} \delta & x > 0 \\ 0 & x \le 0 \end{cases}$	_____
Sigmoid	$\dfrac{\partial L}{\partial x} = \delta \cdot y(1-y)$	_____

(4) **代码实现**：用 Python/NumPy 实现 ReLU 层的 `forward` 和 `backward` 方法，要求 `backward` 利用 `forward` 阶段保存的信息：

```python
import numpy as np

class ReLU:
    def __init__(self):
        self.mask = None    # 记录哪些位置被置零

    def forward(self, x):
        # 请补全
        pass

    def backward(self, dout):
        # 请补全（dout 是上游梯度）
        pass

```


来源溯源：[文档第 4 章 4.4 激活层的反向传播和实现]

小白必读：为什么激活函数的导数如此重要？

- **激活层是梯度必经之路**：梯度从 Loss 出发，每经过一个激活层，就要乘以该激活函数的导数。如果导数很小（如 Sigmoid 在饱和区导数趋近于 0），梯度经过多层后就会消失。
- **ReLU 的"开关"特性**：前向传播时，ReLU 把负输入变成 0（开关关闭），正输入原样通过（开关打开）。反向传播时，开关关闭的神经元梯度直接变成 0（那个方向的信号被彻底截断），开关打开的神经元梯度原样通过——干净利落，不引入额外衰减。
- **Sigmoid 导数的上限**：$y(1-y)$ 在 $y=0.5$ 时取最大值 0.25，且当 y 接近 0 或 1 时（饱和区），导数趋近于 0。这正是 Sigmoid 引发梯度消失的根本原因。

参考答案与详细解析

(1) ReLU 的反向传播：

前向传播：$y=\max(0,x)$，故局部导数为：

$$\frac{\partial y}{\partial x}=\begin{cases}1 & x>0\\ 0 & x\le 0\end{cases}$$

由链式法则：

$$\frac{\partial L}{\partial x}=\delta\cdot\frac{\partial y}{\partial x}=\begin{cases}\delta & x>0\\ 0 & x\le 0\end{cases}$$

"开关"描述：前向时 $x>0$ 的位置"开关打开"，梯度原样通过；$x\le 0$ 的位置"开关关闭"，梯度直接置零。开关的状态（哪些位置为正）在前向传播时记录，反向传播时直接复用。

(2) Sigmoid 的反向传播：

$$\frac{\partial L}{\partial x}=\delta\cdot y(1-y)$$

当 $y=0.880$，$\delta=1$ 时：

$$\frac{\partial L}{\partial x}=1\times 0.880\times(1-0.880)=0.880\times 0.120=\boxed{0.106}$$

(3) 梯度特点对比：

激活函数	反向传播公式	梯度特点
ReLU	δ（$x>0$）或 0（$x\le 0$）	正值区域梯度恒为 1，不引入额外衰减，可有效缓解梯度消失；但负值区域梯度恒为 0（死亡 ReLU 风险）
Sigmoid	$\delta\cdot y(1-y)$	导数最大值仅 0.25，饱和区趋近于 0，多层连乘后梯度快速消失，不适合深层网络隐藏层

(4) ReLU 层代码实现：

```python
import numpy as np

class ReLU:
    def __init__(self):
        self.mask = None  # 记录前向传播中哪些位置 <= 0

    def forward(self, x):
        self.mask = (x <= 0)          # True 表示该位置被 ReLU 置零
        out = x.copy()
        out[self.mask] = 0            # 负值和零位置输出置零
        return out

    def backward(self, dout):
        dout[self.mask] = 0           # 前向被置零的位置，梯度也置零
        dx = dout
        return dx

```

前向保存信息供反向使用——这是层实现的通用模式

- **ReLU**：保存 mask（哪些位置 ≤ 0），反向时直接用
- **Sigmoid**：保存前向输出 y，反向时计算 $y(1-y)$（无需重新算指数）
- **乘法节点**：分别保存 x 和 y，反向时互换使用

Q4-05 Affine 层（全连接层）的反向传播

题目描述：

文档第 4.5 节推导了 Affine 层（即全连接层）在**批量输入**下的反向传播。Affine 层是 $Y = XW + b$，当 X 是一个批量矩阵时，梯度的推导需要特别注意形状。

设批量大小为 $N = 2$，输入特征数 $D = 3$，输出特征数 $M = 4$：

$$X_{(N\times D)} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad W_{(D\times M)} = \begin{pmatrix} w_{11} & \cdots \\ \vdots & \ddots \end{pmatrix}, \quad b_{(1\times M)} = \begin{pmatrix} b_1 & b_2 & b_3 & b_4 \end{pmatrix}$$

前向传播：$Y = XW + b$，上游传来的梯度为 $\dfrac{\partial L}{\partial Y}$（形状与 Y 相同，记为 δ_Y，形状 $N \times M$）。

问题：

(1) 写出 Y 的形状（$N \times M$）。

(2) 推导以下三个梯度（提示：维度分析是最快的推导方法——保证等式两边的形状一致）：

a. $\dfrac{\partial L}{\partial W}$（形状应为 $D \times M$，与 W 相同）

b. $\dfrac{\partial L}{\partial b}$（形状应为 $1 \times M$，与 b 相同）

c. $\dfrac{\partial L}{\partial X}$（形状应为 $N \times D$，与 X 相同）

(3) **形状验证**：给出推导结果后，验证以下等式的形状一致性：

$$\frac{\partial L}{\partial W} = X^T \delta_Y : \quad (D \times N)(N \times M) = D \times M \checkmark$$

$$\frac{\partial L}{\partial X} = \delta_Y W^T : \quad (N \times M)(M \times D) = N \times D \checkmark$$

$$\frac{\partial L}{\partial b} = \sum_{n=1}^{N} \delta_{Y,n} : \quad \text{对} N \text{ 个样本的梯度求和，得} 1 \times M \checkmark$$

(4) **为什么 b 的梯度要对 N 个样本求和**？从偏置的广播机制和反向传播的梯度分支累加规则解释。

(5) 用 Python/NumPy 实现 Affine 层的 `forward` 和 `backward` 方法：

```python
import numpy as np

class Affine:
    def __init__(self, W, b):
        self.W  = W
        self.b  = b
        self.x  = None   # 保存前向输入

    def forward(self, x):
        # 请补全
        pass

    def backward(self, dout):
        # 请补全，返回 dx，并更新 self.dW, self.db
        pass

```

小白必读：矩阵求导怎么不出错？用形状来推！

- **维度分析法**：对于矩阵运算的反向传播，最可靠的方法不是死记公式，而是**让等式两边的形状匹配**。例如，$\partial L/\partial W$ 的形状必须和 W 一样是 $D\times M$，能凑出这个形状的只有 $X^T\delta_Y$（$(D\times N)(N\times M)$）。
- **转置的直觉**：矩阵乘法 $Y = XW$ 中，前向传播 X 在左；反向传播计算 $\partial L/\partial W$ 时，X 变成 X^T 在左。这个"转置换位置"的模式在所有矩阵乘法的反向传播中都成立。
- **偏置的特殊处理**：偏置 b 在前向传播时通过广播加到每个样本上（N 次），相当于 N 条路径共用同一个 b，反向传播时 N 条路径的梯度需要全部加回给 b——所以要对样本维度求和。

参考答案与详细解析

(1) Y 的形状：

$Y = XW + b$，X 形状 $(N\times D) = (2\times 3)$，$W$ 形状 $(D\times M) = (3\times 4)$，故 Y 形状为 $\boxed{N\times M = 2\times 4}$。

(2) 三个梯度的推导：

(a) $\partial L/\partial W$（形状 $D\times M$）：

由形状匹配，唯一能从 X（$(N\times D)$）和 δ_Y（$(N\times M)$）凑出 $D\times M$ 的是：

$$\frac{\partial L}{\partial W} = X^T\delta_Y \quad (D\times N)(N\times M) = D\times M$$

(b) $\partial L/\partial b$（形状 $1\times M$）：

b 对每个样本都广播了一次，每次前向传播产生一条梯度路径，反向时求和：

$$\frac{\partial L}{\partial b} = \sum_{n=1}^{N}\delta_{Y,n} = \mathbf{1}^T\delta_Y \quad (1\times N)(N\times M) = 1\times M$$

即对 δ_Y 按行求和（`np.sum(dout, axis=0)`）。

(c) $\partial L/\partial X$（形状 $N\times D$）：

$$\frac{\partial L}{\partial X} = \delta_Y W^T \quad (N\times M)(M\times D) = N\times D$$

(3) 形状验证：已在题目中给出，三式均验证通过。✓

(4) 偏置梯度求和的原因：

前向传播中，b 通过广播同时加到了 N 个样本的输出上，相当于在计算图中，b 这个节点同时连接到了 N 个"加法节点"（每个样本对应一个）。根据分支节点的梯度累加规则，b 从 N 条路径各接收一份梯度，总梯度 = 各路径梯度之**和**，即对批量维度（axis=0）求和。

(5) Affine 层代码实现：

```python
import numpy as np

class Affine:
    def __init__(self, W, b):
        self.W  = W
        self.b  = b
        self.x  = None
        self.dW = None
```

```
        self.db = None

    def forward(self, x):
        self.x = x                          # 保存输入供反向使用
        out = x @ self.W + self.b           # Y = XW + b
        return out

    def backward(self, dout):
        dx        = dout @ self.W.T         # dL/dX =   W^T
        self.dW   = self.x.T @ dout         # dL/dW = X^T
        self.db   = np.sum(dout, axis=0)    # dL/db = 各样本梯度求和
        return dx

```

Q4-06 综合代码实现：两层网络的完整反向传播

题目描述：

文档第 4.7 节给出了一个完整的反向传播综合实现。以下是一个简化的两层神经网络结构，包含 Affine 层、ReLU 激活层和 Softmax-CrossEntropy 输出层：

```python
class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size):
        # 初始化权重（已省略）
        self.layers = {
            'Affine1': Affine(W1, b1),
            'ReLU1':   ReLU(),
            'Affine2': Affine(W2, b2),
        }
        self.last_layer = SoftmaxWithLoss()

    def predict(self, x):
        for layer in self.layers.values():
            x = layer.forward(x)
        return x

    def loss(self, x, t):
        y = self.predict(x)
        return self.last_layer.forward(y, t)

    def gradient(self, x, t):
        # 前向传播
        self.loss(x, t)

        # 反向传播
        dout = 1                                    # 步骤A
        dout = self.last_layer.backward(dout)       # 步骤B
        layers = list(self.layers.values())
        layers.reverse()
        for layer in layers:
            dout = layer.backward(dout)             # 步骤C

        # 提取各层梯度
        grads = {
            'W1': self.layers['Affine1'].dW,
            'b1': self.layers['Affine1'].db,
            'W2': self.layers['Affine2'].dW,
            'b2': self.layers['Affine2'].db,
        }
        return grads

```

问题：

(1) **前向传播数据流**：predict 方法中，数据依次经过哪些层？从输入 x 到最终 logits 的变换链是什么（用符号表示）？

小白必读：为什么把 Softmax 和 CrossEntropy 合并成一个层？

- **数值稳定性**：Softmax 分子分母中有 e^{z_k}，当 z_k 很大时会数值溢出。合并后可以用"log-sum-exp"技巧在对数域计算，避免中间步骤的数值溢出。
- **梯度极简**：分开计算时，CrossEntropy 对 Softmax 输出的梯度要乘以 Softmax 自身的雅可比矩阵，形式复杂。合并后，两者的复杂项恰好约分，最终梯度简化为 $y_k - t_k$，既简洁又高效。
- **实践中对应**：PyTorch 的 `nn.CrossEntropyLoss` 就是 `LogSoftmax + NLLLoss` 的合并版，必须接收 logits（未经 Softmax 的原始输出）而不是概率，原因正在于此。

参考答案与详细解析

(1) 前向传播数据流：

数据依次经过：`Affine1` → `ReLU1` → `Affine2`。符号表示：

$$x \xrightarrow{XW_1+b_1} a_1 \xrightarrow{\text{ReLU}} h_1 \xrightarrow{h_1W_2+b_2} \text{logits}$$

`loss` 方法额外经过 `SoftmaxWithLoss`，计算最终损失值。

(2) 步骤 A `dout=1` 的含义：

$\dfrac{\partial L}{\partial L} = 1$ 是恒等式——Loss 对自身的导数等于 1。这是反向传播的起点：最终标量 Loss 对自身的梯度为 1，相当于告诉第一个反向节点" 整体的权重是 1"，之后链式法则会自动将这个"1" 按因果比例分配给每一层。若将 Loss 乘以某个系数（如取平均），这里就应该填入相应系数。

(3) SoftmaxWithLoss 反向传播的直觉含义：

梯度 $y_k - t_k$ 的直觉：

- 若 $y_k = t_k$（预测完全正确）：梯度 $= 0$，无需调整；
- 若 $y_k > t_k$（预测概率偏高）：梯度 > 0，应减小 z_k（降低该类别的" 分数"）；
- 若 $y_k < t_k$（预测概率偏低）：梯度 < 0，应增大 z_k（提高该类别的" 分数"）。

当预测与标签完全吻合时，梯度为**零向量**，参数不再更新——模型已经找到了" 完美答案"，停止学习，这正是我们期望的行为。

(4) 反向顺序的必要性：

反向传播是链式法则的逐节点展开。计算某层的梯度时，需要" 上游（更接近 Loss 的方向）已经算好的梯度"。若不按反向顺序，某层的上游梯度尚未计算完毕，就无法得到正确的传入梯度，链式法则断裂。因此，必须从 Loss 出发，严格按照前向传播的逆序依次调用 `backward`。

(5) 数值微分的原理及局限：

数值微分利用导数定义的近似（中心差分，误差 $O(h^2)$）：

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\ldots, x_i + h, \ldots) - f(\ldots, x_i - h, \ldots)}{2h}$$

代码对每个参数 x_i 分别加减一个小量 $h = 10^{-4}$，调用两次前向传播，用差商近似偏导数。

为什么实际训练不用数值微分：

- **计算量极大**：对每个参数都需要两次前向传播。一个拥有 10^7 个参数的网络，需要 2×10^7 次前向传播——实际训练中不可接受；
- **反向传播只需一次**：一次前向 + 一次反向传播，就能同时算出所有参数的梯度，效率高出数量级；

- **数值微分的正确用途：梯度检验（Gradient Check）**——在开发新层时，用数值微分验证反向传播实现是否正确，而非用于实际训练。

梯度检验的工作流程

(1) 实现层的 `forward` 和 `backward`；

(2) 用数值微分计算”近似梯度”；

(3) 用 `backward` 计算”解析梯度”；

(4) 比较两者差异（通常要求相对误差 $< 10^{-5}$）；

(5) 若相差很大，说明 `backward` 实现有 bug。

现实应用场景

反向传播——60 年前的思想，今天的核心引擎：
反向传播算法的核心思想最早可追溯到 1970 年代，但直到 1986 年被引入神经网络后才真正改变世界。今天，PyTorch、TensorFlow 等框架的自动微分系统，本质上都是”动态计算图 + 反向传播”——你写的每一行前向计算代码，框架都在后台悄悄记录计算图，一旦调用 `.backward()`，梯度就沿图反向流动，自动填入每个参数的 `.grad`。理解本章，相当于理解了深度学习框架最核心的那台”发动机”是如何工作的。

2 本章小结：反向传播的全景

学完这 6 道题，如果还觉得”原理飘在空中”，试试把整章内容浓缩成这几句画面：

- **计算图** = 把复杂计算拆成只做一件事的小节点，每个节点只需要知道自己的局部导数；
- **链式法则** = 梯度的传递方式：上游梯度 × 局部导数 = 传给下游的梯度；分支处梯度相加；
- **加法节点** = 梯度原样广播；**乘法节点** = 梯度乘以对方的前向值（互换）；
- **ReLU** = 梯度的开关，正值区原样通过，负值区直接截断；**Sigmoid** = 梯度最大只有 0.25，多层后迅速消失；
- **Affine 层** = $\partial L/\partial W = X^T\delta_Y$，$\partial L/\partial X = \delta_Y W^T$，$\partial L/\partial b$ = 对样本维度求和；
- **完整流程** = 前向算 Loss，反向从 1 出发按逆序逐层调用 `backward`，梯度自动流到每个参数。

给零基础读者的一句总比喻

如果把神经网络比作一条生产流水线：

- **前向传播** = 原材料从左到右经过每道工序，最终生产出产品；
- **Loss** = 质检员打出的" 不合格分数"；
- **反向传播** = " 责任追溯"——质检报告从右到左传递，每道工序根据自己的" 参与比例"（局部导数），承担相应的责任（梯度）；
- **梯度下降** = 每道工序根据自己承担的责任，调整操作参数，下次少出错；
- **自动微分框架** = 一套自动记录每道工序参与比例的管理系统——你只需要设计工序（前向计算），责任追溯（反向传播）全自动完成。